

Risk Modelling Activities: Monte Carlo Simulation and Bayesian Analysis

Abinel Madzivire

Introduction

This activity developed my understanding of quantitative risk modelling through practical Python exercises involving Monte Carlo simulation and Bayesian probability. I worked through Fizell's (2022) Monte Carlo simulation exercise and then used Downey's *Think Bayes 2* notebooks to complete the exercises in Chapters 1 and 2. The activities demonstrated how Python can be used to model uncertainty, calculate conditional probabilities and update probability estimates when new evidence becomes available.

Monte Carlo Simulation

The first activity involved working through Fizell's (2022) tutorial on creating a Monte Carlo simulation using Python. Monte Carlo simulation uses repeated random sampling to model uncertainty and estimate the probability of different outcomes.

The screenshot shows the PyCharm IDE interface. The main editor window displays a Python script named `script.py` with the following code:

```
1
2 import random
3 def roll_dice(): usage
4     return random.randint(a=1, b=6)
5 results = []
6 for i in range(10000):
7     result = roll_dice()
8     results.append(result)
9     print(results)
```

The bottom panel shows the Run console window, which displays the output of the script. The output is a single line containing 10,000 integers, representing the results of the random rolls. The output is truncated in the image, showing only the first and last few values.

The exercise demonstrated how a random process can be simulated repeatedly in Python. Rather than relying on the result of a single trial, many simulations can be performed and the resulting distribution analysed. This provides a more useful representation of uncertainty.

From a cybersecurity risk perspective, I found this particularly relevant because risk assessments frequently involve uncertain variables. For example, the frequency of cyber incidents, duration of system downtime and financial consequences of a data breach cannot normally be predicted precisely. Monte Carlo simulation provides a method of representing these variables probabilistically and examining a range of possible outcomes.

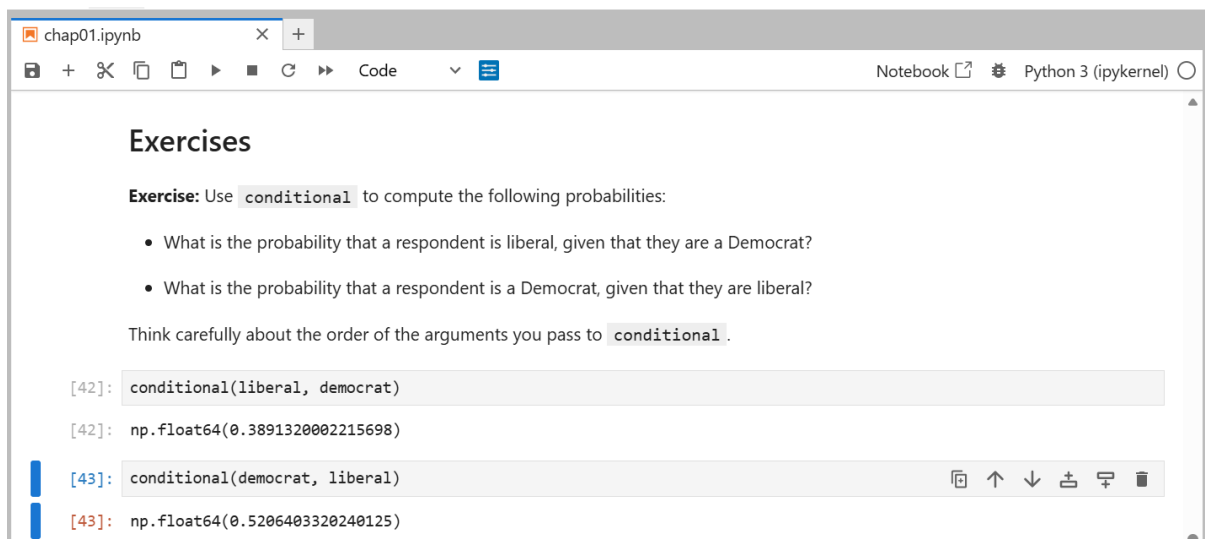
An important learning point was that increasing the number of simulations can improve the stability of the estimated results, but this does not automatically make the underlying model accurate. The quality of a simulation remains dependent upon the assumptions, probability distributions and data used as inputs. Therefore, quantitative results still require critical interpretation.

Think Bayes Chapter 1: Probability

Chapter 1 of Downey's *Think Bayes 2* introduced probability through practical analysis of General Social Survey data. The notebook required the dataset to be loaded into a Pandas DataFrame before Boolean variables such as `banker`, `female`, `liberal` and `democrat` could be created.

The exercises helped me understand the distinction between conjunction and conditional probability. For example, I used the `conditional()` function to investigate the probability that a respondent was liberal given that they were a

Democrat and then reversed the conditions to calculate the probability that a respondent was a Democrat given that they were liberal.



The screenshot shows a Jupyter Notebook interface with a tab labeled 'chap01.ipynb'. The notebook contains a section titled 'Exercises' with the following text: 'Exercise: Use `conditional` to compute the following probabilities:

- What is the probability that a respondent is liberal, given that they are a Democrat?
- What is the probability that a respondent is a Democrat, given that they are liberal?

Think carefully about the order of the arguments you pass to `conditional`.

The notebook shows two code cells. The first cell, labeled '[42]:', contains the code `conditional(liberal, democrat)` and its output is `np.float64(0.389132002215698)`. The second cell, labeled '[43]:', contains the code `conditional(democrat, liberal)` and its output is `np.float64(0.5206403320240125)`.

This demonstrated that:

$P(\text{Liberal} \mid \text{Democrat})$ is not necessarily equal to $P(\text{Democrat} \mid \text{Liberal})$.

This distinction is important because the conditioning information determines the population against which the probability is calculated.

The chapter also applied conditional probability to a variation of the Linda problem. I compared the probability that a female respondent was a banker with the probability that a female respondent was simultaneously a banker, liberal and Democrat. This demonstrated the conjunction rule: requiring several conditions to be true simultaneously normally produces a smaller probability than requiring only one condition.

```
chap01.ipynb
[44]: conditional(banker, female)
[44]: np.float64(0.02116102749801969)
[45]: conditional(banker & liberal & democrat, female)
[45]: np.float64(0.0023009316887329786)
```

The final exercises introduced age and political-view variables, including `young`, `old`, `liberal` and `conservative`. These exercises reinforced the difference between questions involving conjunctions, such as the proportion of respondents who were both young and liberal, and conditional questions, such as the proportion of young respondents who were liberal.

Completing these exercises improved my understanding of how the wording of a probability question determines the appropriate Python operation. I also learnt the importance of executing Jupyter Notebook cells sequentially because later variables and functions depend upon objects created in earlier cells.

Think Bayes Chapter 2: Bayes's Theorem

Chapter 2 extended the concepts from Chapter 1 by introducing Bayes's theorem. The key concept was Bayesian updating, where an initial probability or **prior** is modified in response to observed evidence to produce a **posterior probability**.

The exercises used Bayes tables to organise the calculation into hypotheses, prior probabilities, likelihoods, unnormalised probabilities and posterior probabilities.

The first exercise considered two coins: one normal coin and one double-headed trick coin. Both coins initially had equal prior probabilities. After observing heads, the posterior probability of the trick coin increased because the observed evidence was more likely under the trick-coin hypothesis. The resulting probability of having selected the trick coin was approximately 66.7%.

The second exercise considered a family with two children. Starting with four equally likely hypotheses—BB, BG, GB and GG—the evidence that at least one child was a girl eliminated BB. Under the assumptions of the exercise, the remaining possibilities were equally likely, giving a probability of $1/3$ (approximately 33.3%) that both children were girls.

The third exercise examined a variation of the Monty Hall problem in which Monty's behaviour followed a specified rule. This demonstrated an important aspect of Bayesian reasoning: evidence cannot be interpreted independently of the process that generated it. Monty's decision about which door to open provides information because his behaviour depends upon the location of the car.

The final exercise involved M&M colour distributions from 1994 and 1996. Two competing hypotheses were constructed concerning which bag produced a yellow M&M and which produced a green M&M. The likelihood of the observed colours was calculated under each hypothesis and combined with the prior probabilities. The posterior probability that the yellow M&M came from the 1994 bag was approximately 74.1%. This exercise provided a particularly clear demonstration of how evidence can alter the relative probability of competing hypotheses.

Exercise: M&M's are small candy-coated chocolates that come in a variety of colors. Mars, Inc., which makes M&M's, changes the mixture of colors from time to time. In 1995, they introduced blue M&M's.

- In 1994, the color mix in a bag of plain M&M's was 30% Brown, 20% Yellow, 20% Red, 10% Green, 10% Orange, 10% Tan.
- In 1996, it was 24% Blue, 20% Green, 16% Orange, 14% Yellow, 13% Red, 13% Brown.

Suppose a friend of mine has two bags of M&M's, and he tells me that one is from 1994 and one from 1996. He won't tell me which is which, but he gives me one M&M from each bag. One is yellow and one is green. What is the probability that the yellow one came from the 1994 bag?

Hint: The trick to this question is to define the hypotheses and the data carefully.

```
[15]: table = pd.DataFrame(index=['A', 'B'])
table['prior'] = 1/2
table['likelihood'] = [
    0.20 * 0.20,
    0.14 * 0.10
]
table['unnorm'] = table['prior'] * table['likelihood']
prob_data = table['unnorm'].sum()
table['posterior'] = table['unnorm'] / prob_data
table
```

```
[15]:
```

	prior	likelihood	unnorm	posterior
A	0.5	0.040	0.020	0.740741
B	0.5	0.014	0.007	0.259259

Think Bayes, Second Edition

Application to Cybersecurity Risk Modelling

The exercises have direct relevance to cybersecurity. Monte Carlo simulation can be used to model uncertain cyber-risk variables such as incident frequency, financial loss, recovery time and service downtime. Instead of producing a single deterministic estimate, the technique can generate a distribution of possible losses and their associated probabilities.

Bayesian analysis provides a complementary approach because cybersecurity assessments frequently change when new evidence becomes available. For example, an organisation might initially assess the probability of system compromise as relatively low. Evidence from vulnerability scans, failed authentication attempts, unusual network traffic, endpoint alerts or threat intelligence could subsequently increase or decrease that probability.

Bayesian reasoning therefore provides a structured approach for moving from an initial risk assessment to an updated assessment based on evidence. This is particularly

relevant to security monitoring and incident investigation, where decisions are often made incrementally as information becomes available.

Monte Carlo and Bayesian methods can also complement each other. Bayesian analysis can be used to update probability estimates as evidence changes, while Monte Carlo simulation can use probability distributions to explore the range of possible consequences.

Reflection

Completing these activities strengthened both my Python skills and my understanding of quantitative risk modelling. Initially, I encountered practical challenges when working with Jupyter, including variables such as `gss`, `female` and `banker` not being defined. Resolving these problems helped me understand that Jupyter Notebook cells must normally be executed sequentially because later calculations depend upon variables, functions and datasets established earlier in the notebook.

The Chapter 1 exercises improved my understanding of conjunction and conditional probability, particularly the importance of correctly identifying which event represents the condition. Chapter 2 then demonstrated how prior probabilities can be updated systematically when new evidence becomes available.

The Monte Carlo activity provided a different but complementary perspective by demonstrating how repeated random simulation can represent uncertainty across a range of possible outcomes. Together, these activities showed that quantitative risk modelling is not intended to eliminate uncertainty. Instead, it provides structured methods for representing, analysing and communicating uncertainty so that risk decisions can be better informed.

From a professional cybersecurity perspective, I can see potential applications in vulnerability management, incident analysis, network security and quantitative cyber-risk assessment. The activity has therefore helped me connect probability theory and Python programming with practical information-security risk management.

Evidence for my e-portfolio

Evidence retained from this activity includes the completed Chapter 1 and Chapter 2 Jupyter notebooks, Python code and outputs from the Monte Carlo simulation, conditional probability calculations from Chapter 1, completed Bayes tables from Chapter 2, and screenshots showing successful execution of selected exercises. These artefacts demonstrate practical engagement with quantitative risk-modelling techniques rather than theoretical understanding alone.

References

Downey, A.B. (2021) *Think Bayes: Bayesian Statistics in Python*. 2nd edn. Sebastopol, CA: O'Reilly Media.

Fizell, Z. (2022) 'How to create a Monte Carlo simulation using Python', *Towards Data Science*. Available at: <https://towardsdatascience.com/> (Accessed: 19 September 2026).